PixelPine Implementation Plan

> Executed inline in the authoring session (user asked for minimal interruptions).

Goal: Produce every deliverable in the spec: KiCad schematic + placed PCB, BOM, browser simulator, market / cost / power / EU-compliance docs.

Architecture: One Python geometry module is the single source of truth for the tree outline, LED positions and the charlieplex map. Generators consume it to write the KiCad schematic (s-expression text) and the PCB (KiCad's bundled `pcbnew` Python API), and to export `led_map.json` for the simulator. The simulator is a single static HTML file.

Tech Stack: KiCad 10.0 (`kicad-cli`, bundled Python 3.11 + `pcbnew`), vanilla HTML/Canvas/JS, Markdown.

Spec: `docs/superpowers/specs/2026-09-23-pixelpine-design.md`

Global Constraints

- 240 LEDs = 16×15 charlieplex on PY32F003F18P6 PA0–PA6, PA12, PA13, PA14, PB5–PB7, PF0, PF1, PF4; touch on PA7 (ADC); NRST kept; SWD programming.
- LED id `n` ↔ (anode a, cathode c): a = n mod 16, c = (a + 1 + n div 16) mod 16 (interleaved phases). *Superseded:* the pair at each position now comes from `hardware/led_assignment.json`, chosen so the board routes on 2 layers (see the README).
- Total tree height 70 mm incl. trunk; top-side-only SMT; lead-free HASL.
- LCSC parts: C5187096, C965810, C25076, C25905, C456012, C19702, C1525.

Task 1: Geometry + LED map — `hardware/scripts/geometry.py`

- Produces `outline_polygon() -> list[(x_mm, y_mm)]`, `led_positions() -> list[(x, y, rot_deg)]` (exactly 240, sorted top→bottom, left→right), `charlie(n) -> (a, c)`, `foot_outline()`, `STAR_POLY`, and constants.
- Writes `hardware/led_map.json` `{ "leds": [{"id","x","y","a","c"}], "outline": [...], "star": [...] }`.
- Verify: script asserts 240 LEDs, all inside the outline with ≥0.9 mm clearance, and min spacing ≥ 2.2 mm.

Task 2: Schematic — `hardware/scripts/gen_sch.py` → `hardware/pixelpine.kicad_sch`

- Embeds lib symbols from KiCad stock libs (project PY32F003F18P6 symbol, LED, R, C, USB_C_Receptacle_PowerOnly_6P, TestPoint, power symbols).
- Two logical areas: MCU/power/programming, and the 16×15 LED matrix drawn as a grid with net labels L0..L15.
- Verify: `kicad-cli sch erc` → 0 errors; `kicad-cli sch export pdf` + svg render inspected.

Task 3: PCB — `hardware/scripts/gen_pcb.py` → `hardware/pixelpine.kicad_pcb`

- Board outline (tree + foot, same panel), slots, footprints placed with nets from the same map, star pad, test pads, silkscreen.
- Verify: `kicad-cli pcb render` / `export svg` images inspected; netlist consistency checked by script.

Task 4: Simulator — `simulator/index.html`

- Animation interface: `{ name, init(ctx) , frame(ctx, t, dt, out:Float32Array(240)) }`, ctx has `leds` (x,y normalized), `rand()`, helpers.
- Hardware model: BCM bit depth, gamma, current sag per anode phase, refresh.
- Verify in the built-in browser: every animation renders, no console errors, playlist + touch-star cycle works.

Task 5: Docs — BOM, cost, market, power, EU compliance

- BOM prices verified live on LCSC; cost model at 100 / 1k / 10k.

- Verify: numbers in cost doc reconcile with BOM (script-generated tables).